PackTrax — How It Works

A guided tour of the code, for learning purposes

Contents

1. What PackTrax Is **2**

2. External Packages and Services **2**
2.1 Leaflet.js — interactive maps . . . 2
2.2 Google Fonts — Oswald, Inter, JetBrains Mono . . . 2
2.3 Map tile providers (OpenStreetMap / Esri / Carto) . . . 3
2.4 OSRM — road-snapping (routing.openstreetmap.de) . . . 3
2.5 Firebase — cloud storage (deployed app only) . . . 3
2.6 Browser Web APIs (not installed, just used) . . . 4

3. Overall Code Shape **4**
3.1 The `state` object . . . 4
3.2 Loose `let` variables for everything else . . . 4
3.3 The render loop . . . 5

4. The Track Tab **5**
4.1 Starting a walk . . . 5
4.2 GPS tracking . . . 6
4.3 Stop → Review → Resume/Save (not Discard) . . . 6
4.4 Crash recovery (autosave) . . . 6
4.5 Daycare / Boarding check-in . . . 7

5. The History Tab **7**
5.1 One merged, sorted list . . . 7
5.2 Expand/collapse . . . 7
5.3 Editing . . . 7
5.4 Share snapshots . . . 8

6. The Invoice Tab **9**
6.1 Treating three record types as one list . . . 9
6.2 Selection and pricing . . . 9
6.3 Payment links in the shared text . . . 9

7. Persistence Layer (Firestore) **9**

8. The PWA Shell **10**

9. Honest Limitations Worth Knowing **10**

1. What PackTrax Is

PackTrax is a dog-walking/daycare/boarding business app. It tracks walks with GPS, manages daycare and boarding check-ins, keeps a history log, and builds invoices.

It is built as a **single HTML file** — one file contains the page structure (`<head>`/`<body>`), all the visual design (one big `<style>` block), and all the logic (one big `<script>` block). There is no build step, no bundler, no npm install, and no framework like React or Vue. Every screen is plain JavaScript building HTML text and dropping it into the page.

Two versions of this one file exist:

- **The chat artifact** (what you see in Claude) — saves data using a Claude-only `window.storage` API that only works inside Claude's interface.
- **The deployed web app** (what's in your GitHub Pages / zip) — saves data using **Firebase Firestore** instead, so it works as a real, hosted website.

Everything else about the two files is identical.

---

2. External Packages and Services

PackTrax doesn't use `npm install` or a `package.json` — in a single static HTML file, "installing a package" means **loading it from a CDN with a `<script>` tag**. Here's everything it pulls in, and what each one is for.

2.1 Leaflet.js — interactive maps

```
<link rel="stylesheet" href="https://cdnjs.cloudflare.com/.../leaflet.min.css">
<script src="https://cdnjs.cloudflare.com/.../leaflet.min.js"></script>
```

Leaflet is the library behind every **on-screen, interactive** map in the app: the live map while tracking a walk, the map shown on an expanded walk card in History, and the draggable-point route editor. It handles:

- Drawing map tiles (the street imagery itself)
- Drawing your route as a polyline on top
- Letting you pan/zoom
- Draggable markers (used by the route editor)

Leaflet itself doesn't supply map *imagery* — it just draws whatever tiles you point it at (see 2.3).

2.2 Google Fonts — Oswald, Inter, JetBrains Mono

```
<link href="https://fonts.googleapis.com/css2?family=Oswald:...&family=Inter:...&family=JetBrai
```

Three typefaces, each used for a different purpose:

- **Oswald** — condensed, bold headers (trail-sign feel)
- **Inter** — regular body text
- **JetBrains Mono** — a monospace font used wherever a *number* needs to stand out: timers, distances, prices, durations

2.3 Map tile providers (OpenStreetMap / Esri / Carto)

These aren't JavaScript libraries — they're **image servers**. A "tile" is just a 256×256 pixel PNG of a small square of the map at a given zoom level; Leaflet (or plain canvas code) requests dozens of them and stitches them together.

Three different tile sources show up in the code, for two different jobs:

Context	Source	Why
Live tracking map, History map, route editor	`tile.openstreetmap.org`	Standard free OSM tiles, fine for on-screen `<img>`-style display
Share-snapshot images (walk/daycare/boarding photos)	`server.arcgisonline.com` (Esri), falling back to `basemaps.cartocdn.com` (Carto)	These specific sources send the CORS header a `<canvas>` needs to actually *read pixels* from a tile image, which standard OSM tiles usually don't

That second row matters because generating a shareable *image* works differently from just displaying a map. A browser `<canvas>` is allowed to draw a cross-origin image, but it refuses to let you later export that canvas (`toBlob()`) unless the image was loaded with explicit permission (`crossOrigin="anonymous"` + a CORS header from the server). This was the exact bug behind the "map isn't showing on shares" issue — the original tile source wasn't sending that header, so every tile silently failed to load, leaving a blank background behind the route line.

2.4 OSRM — road-snapping (routing.openstreetmap.de)

A free, public routing/map-matching service. After you save a walk with a GPS route, PackTrax sends the recorded points to:

`https://routing.openstreetmap.de/routed-foot/match/v1/foot/{coords}?geometries=geojson&overview`

This "matches" your noisy phone-GPS trace onto the actual street/path network, so two points with a weak-signal gap get connected by a real road instead of a straight line cutting through buildings. It runs **after** save (not live) and updates the stored route once it responds — see Section 6.4.

2.5 Firebase — cloud storage (deployed app only)

```
<script src="https://www.gstatic.com/firebasejs/.../firebase-app-compat.js"></script>
<script src="https://www.gstatic.com/firebasejs/.../firebase-auth-compat.js"></script>
<script src="https://www.gstatic.com/firebasejs/.../firebase-firestore-compat.js"></script>
```

Three pieces, used together:

- **Firebase App** — initializes the connection to your project using `firebaseConfig` (the keys you provided)
- **Firebase Auth** — signs the browser in **anonymously** on load, so Firestore's security rules can require "must be signed in" without you needing to build a login screen
- **Firestore** — a cloud NoSQL database. PackTrax stores each data type (dogs, walks, daycare, boarding, plus the in-progress-walk snapshot) as **one document per type**, each holding the full array as a single field

2.6 Browser Web APIs (not installed, just used)

These ship with every modern browser — nothing to load — but they're worth knowing about since they power several features:

- **Geolocation API** (`navigator.geolocation.watchPosition`) — GPS tracking
- **Wake Lock API** (`navigator.wakeLock`) — keeps the screen from sleeping mid-walk, since GPS updates pause once the screen locks
- **Web Share API** (`navigator.share`) — hands an image + text to the phone's native share sheet (Messages, etc.)
- **Canvas API** — draws the shareable snapshot images pixel-by-pixel
- **`localStorage`** — used for the migration path, and as a general browser storage primitive

---

3. Overall Code Shape

Open the file and you'll find three big sections, in this order:

```
<head>
  ... <link>/<script> tags for the packages above ...
  <style> ... one large stylesheet ... </style>
</head>
<body>
  <div class="app"> ... tab buttons, #content container ... </div>
  <script>
    ... all application logic lives here ...
  </script>
</body>
```

3.1 The `state` object

```
let state = { dogs: [], walks: [], daycare: [], boarding: [] };
```

This holds every piece of data that gets saved. The four arrays are the "tables" of the app's tiny database. Every other screen reads from and writes to this one object.

3.2 Loose `let` variables for everything else

Dozens of standalone `let` declarations near the top of the script track **session/UI state** — things that matter only while the app is open, not data to be saved:

```
let activeTab = 'track';
let selectedDogIds = [];
let timerRunning = false;
let reviewing = false;
let currentRoute = [];
let editingWalkId = null;
...
```

This is a deliberate (if old-fashioned) pattern: instead of a framework's "component state," the whole app shares one flat pool of variables that any function can read or reassign. It keeps things simple for a project this size, at the cost of needing discipline about resetting variables at the right moments (see Section 4).

3.3 The render loop

```
function render(){
  const container = document.getElementById('content');
  if(activeTab==='track') renderTrack(container);
  else if(activeTab==='dogs') renderDogs(container);
  else if(activeTab==='history') renderHistory(container);
  else if(activeTab==='stats') renderStats(container);
  else if(activeTab==='invoice') renderInvoice(container);
}
```

This is the heart of the UI. Every `render*()` function follows the same pattern:

1. Build up a big string of HTML (`let html = '...'`) based on the current `state` and UI variables
2. Set `container.innerHTML = html` — this **throws away and rebuilds** every DOM element in that tab
3. Immediately after, re-attach event listeners (`addEventListener`) to the freshly created buttons/inputs, since the old elements (and their listeners) were just destroyed

Any time something changes — a button tap, a GPS update, a counter increment — the pattern is almost always: mutate a variable, then call `render()` again. There's no diffing or virtual DOM; it just re-draws the whole tab's HTML from scratch each time. For an app this size, that's fast enough to feel instant.

---

4. The Track Tab

This is the most involved screen. It has three states, controlled by two booleans:

`timerRunning`	`reviewing`	What's shown
`false`	`false`	Dog picker + **Start walk** button
`true`	`false`	Live timer, live map, **Stop walk** button
`true`	`true`	"Reviewing walk" screen — timer/GPS/wake lock are all *still running in the background*

4.1 Starting a walk

```
function startTimer(){
  timerRunning = true;
  timerStart = Date.now() - elapsedMs;
  timerInterval = setInterval(()=>{
    elapsedMs = Date.now() - timerStart;
    document.getElementById('timerDisplay').textContent = fmtTime(elapsedMs);
  }, 500);
  startGeoTracking();
  requestWakeLock();
```

```
  startAutosave();
}
```

The timer doesn't literally count up a number every tick — it stores the **start timestamp** and recomputes `elapsedMs = now - start` every half second. This is more accurate than incrementing a counter, since it survives the browser throttling background timers.

4.2 GPS tracking

```
function startGeoTracking(existingRoute){
  currentRoute = existingRoute ? [...existingRoute] : [];
  geoWatchId = navigator.geolocation.watchPosition(pos => {
    currentRoute.push({ lat: pos.coords.latitude, lng: pos.coords.longitude });
    updateLiveMap(...);
  }, errorHandler, { enableHighAccuracy: true });
}
```

`watchPosition` is a *subscription*, not a one-time read — the browser calls the callback every time it gets a new fix, and each point gets pushed onto `currentRoute`. The `existingRoute` parameter exists so a **resumed** walk (after a crash, or after tapping Resume) can continue appending to the same array instead of starting over.

4.3 Stop → Review → Resume/Save (not Discard)

Tapping **Stop walk** does *not* stop anything — it only flips `reviewing` to `true` and re-renders. The timer interval, the GPS watch, and the wake lock are untouched, so nothing is lost if "Stop" was an accidental tap.

```
function enterReview(){ reviewing = true; render(); }
function resumeWalk(){ reviewing = false; render(); }

function finalizeTracking(){      // the only thing that truly stops tracking
  timerRunning = false;
  clearInterval(timerInterval);
  stopGeoTracking();
  releaseWakeLock();
  stopAutosave();
  clearActiveWalkState();
}
```

`finalizeTracking()` is only ever called from `saveWalk()`. There's no "Discard" path on this screen anymore — if a walk isn't wanted, it still gets saved, and you delete it afterward from History instead (a safer default after losing walks to accidental taps or crashes).

4.4 Crash recovery (autosave)

While a walk is active, a snapshot of its state is written to storage periodically:

```
async function saveActiveWalkState(){
  if(!timerRunning) return;
  const snapshot = { selectedDogIds, elapsedMs, currentRoute, peeCount, poopCount, ... };
  await docRef('activeWalk').set(snapshot);   // Firestore, in the deployed app
}
startAutosave() // sets up setInterval(saveActiveWalkState, 60000)
```

On the next app load, `loadState()` checks for a leftover `activeWalk` document. If one exists, a banner offers **Resume**, **Save as-is**, or **Discard** — this is the one place Discard still exists, since recovering from a crash is a deliberate choice made from a clean screen, not a stray tap mid-walk.

4.5 Daycare / Boarding check-in

The third card on this tab is independent of the walk timer. Tapping **Daycare** or **Boarding** doesn't start anything immediately — it opens an inline confirmation showing the selected dog(s) and a single button that reads **Check in** or **Check out**, decided by whether an *open* record (one with no `checkOutAt`) already exists for that dog:

```
function getActiveDaycareSession(dogId){
  return state.daycare.find(s => s.dogIds.includes(dogId) && !s.checkOutAt);
}
```

Confirming either creates a new record (`checkInAt: now, checkOutAt: null`) or stamps `checkOutAt` on the existing open one. Boarding works identically, plus an extra dropdown (Board with me / Board at home - Check-ins / Board at home - Housesit) that's only shown at check-in time and saved onto the record as `boardingType`.

---

5. The History Tab

5.1 One merged, sorted list

Walks, daycare stays, and boarding stays are three separate arrays in `state`, but History displays them interleaved, newest first:

```
const combined = [
  ...walks.map(w => ({ type:'walk', sortDate:w.date, data:w })),
  ...daycareItems.map(s => ({ type:'daycare', sortDate:s.checkInAt, data:s })),
  ...boardingItems.map(s => ({ type:'boarding', sortDate:s.checkInAt, data:s }))
].sort((a,b) => new Date(b.sortDate) - new Date(a.sortDate));
```

Each item in the loop branches on `type` to decide which card markup to build — a walk card, a daycare card, or a boarding card — but all three share the same outer "collapsible card" shell (date, dog tag, a care-type pastel bubble, a chevron) so the list reads consistently.

5.2 Expand/collapse

```
let expandedWalks = new Set();   // holds the ids of currently-open cards
```

Tapping a card's header toggles its id in this `Set`. Since the render function checks `expandedWalks.has(id)` to decide whether to include a card's detail body in the HTML string at all, a collapsed card's map/edit form/etc. isn't just hidden by CSS — it's never built or mounted, which keeps a long history fast (no map tiles loading for cards you haven't opened).

5.3 Editing

Three independent edit modes exist per walk card, each with its own tracking variable so only one can be open at a time:

- **Edit details** (`editingWalkId`) — pee/poop counters, note, and now start/end time (two `datetime-local` inputs; saving recomputes `durationMs` from the difference)
- **Edit route** (`editingRouteId`) — opens an interactive Leaflet map with a **draggable marker at each GPS point** (`L.marker(..., {draggable:true})`). Dragging a point updates that entry in a working copy of the route array and redraws the line live; Save replaces the walk's stored route and recalculates distance from the edited points
- **Daycare/boarding time edit** (`editingCareId`) — same idea, just for check-in/check-out timestamps

Routes longer than 40 points are downsampled before editing (`downsampleRoute`) — dragging 40 pins on a phone screen is already a lot; a few hundred would be unusable. Saving replaces the *entire* stored route with just the edited (downsampled) points, trading some resolution for an editable result.

5.4 Share snapshots

"Share" doesn't send any app data over the network — it **draws an image** locally using the Canvas API, then hands that image file to the OS share sheet. Each record type has its own builder function (`buildWalkSnapshot`, `buildDaycareSnapshot`, `buildBoardingSnapshot`, `buildInvoiceSnapshot`), but they all follow the same recipe:

```
async function buildWalkSnapshot(w){
  const canvas = document.createElement('canvas');
  const ctx = canvas.getContext('2d');
  ctx.fillStyle = '#ece4cc'; ctx.fillRect(...);      // card background
  ctx.font = '600 22px Oswald'; ctx.fillText(dateStr, ...);  // text, by hand
  // ...draw dog name pills, stats, a map (see below) or a route sketch...
  return new Promise(resolve => canvas.toBlob(resolve, 'image/png'));
}
```

Everything — every pill, every line of text, every number — is positioned with manual `x, y` pixel coordinates (`ctx.fillText(text, x, y)`). There's no layout engine; the code just tracks a running `y` offset and adds to it after each element, which is why you'll see a lot of `y += 34;`-style lines.

For walks with a route, `renderMapCanvas()` fetches real map tiles (Esri, falling back to Carto — see 2.3) and stitches them into a second, offscreen canvas, which then gets drawn into the final snapshot. If every tile fails to load, the function returns `null` and the caller falls back to `drawRouteSketch()` — a simple hand-drawn line on a flat background, so a network hiccup degrades gracefully instead of producing a broken image.

Sharing itself:

```
async function shareWalk(w){
  const blob = await buildWalkSnapshot(w);
  const file = new File([blob], 'walk.png', {type:'image/png'});
  if(navigator.canShare({files:[file]})){
    await navigator.share({ files:[file], title:'...', text:'...' });
  } else {
    downloadBlob(blob, 'walk.png');   // fallback for unsupported browsers
  }
}
```

---

6. The Invoice Tab

Builds a bill from any combination of walks, daycare stays, and boarding stays.

6.1 Treating three record types as one list

```
function getInvoiceableRecords(){
  return [
    ...state.walks.map(r => ({ type:'walk', id:r.id, record:r })),
    ...state.daycare.map(r => ({ type:'daycare', id:r.id, record:r })),
    ...state.boarding.map(r => ({ type:'boarding', id:r.id, record:r }))
  ];
}
```

A handful of small helper functions (`invRecordDate`, `invRecordDurationMs`, `invRecordDurationLabel`, `invRecordTypeLabel`) know how to pull "the date" or "the duration" out of a record *regardless of which type it is* — walks store `date`/`durationMs` directly, while daycare/boarding derive both from `checkInAt`/`checkOutAt`. Everything downstream (the row list, the summary, the snapshot image) works off these helpers instead of checking the type directly each time.

6.2 Selection and pricing

```
let invoiceSelection = new Set();   // ids of checked rows
let invoicePrices = {};             // id -> price string, for the input fields
```

A price typed into a row is written to `invoicePrices` on every keystroke (so the running total updates live) and persisted onto the actual record (`record.price`) only on blur/change, via whichever save function matches that record's type:

```
function invRecordSave(type){
  return type==='walk' ? saveWalks : type==='daycare' ? saveDaycare : saveBoarding;
}
```

Rows with no price set yet default to **$15 (walk) / $25 (daycare) / $50 (boarding)** via `invDefaultPrice(type)`.

6.3 Payment links in the shared text

```
function buildVenmoLink(items){
  const total = items.reduce((a,{price}) => a+price, 0);
  const note = `${dogNames} ${dateRange}`.replace(/\s+/g, '+');
  return `https://venmo.com/tjrichert?txn=pay&amount=${total}&note=${note}`;
}
```

The Venmo deep link is built from the actual selected items — total, dog name(s), and the earliest–latest date range — so it arrives pre-filled on the recipient's end. This link, plus a static Zelle line, gets appended to the share text alongside the invoice image.

---

7. Persistence Layer (Firestore)

Four save functions, one per data type, all following the same shape:

```
async function saveWalks(){
  await docRef('walks').set({ items: state.walks });
}
```

`docRef(key)` resolves to `db.collection('packtrax').doc(key)` — so the whole `walks` array (every walk you've ever logged) lives in **one Firestore document**, not one document per walk. This mirrors how the app already treated data in memory (always load/save the whole array at once), so very little else in the codebase had to change to move off `localStorage`.

```
async function loadState(){
  await ensureFirebaseAuth();          // waits for anonymous sign-in
  state.dogs = await migrateLocalIfEmpty('dogs', 'pack-trail-dogs');
  state.walks = await migrateLocalIfEmpty('walks', 'pack-trail-walks');
  ...
}
```

`migrateLocalIfEmpty()` is a one-time bridge: if Firestore has nothing yet for a given key but the browser's `localStorage` has old data under it, that old data is uploaded once and used immediately — so switching storage backends didn't strand anyone's existing history.

---

8. The PWA Shell

Three extra files make the web app installable on a phone like a native app:

- **`manifest.json`** — name, icons, and `"display": "standalone"`, which is what makes an installed PWA open without browser chrome (no address bar)
- **`sw.js`** (service worker) — intercepts network requests; caches the app's own files so it can still *open* without a connection, while letting live requests (tile images, Firestore, OSRM) pass straight through to the network
- **`icon-192.png` / `icon-512.png`** — the home-screen icon, generated with Python's Pillow library (a simple paw-print drawn with ellipses, not hand-designed artwork)

---

9. Honest Limitations Worth Knowing

- **Share-snapshot maps depend on a third party's CORS headers.** If Esri and Carto both ever stop sending the right headers, map images in shares would silently degrade to the sketch fallback — on-screen maps (Leaflet) are unaffected either way.
- **Road-snapping (OSRM) is a free public service**, not something PackTrax controls or can guarantee uptime for. It's wrapped in a try/catch that quietly keeps the raw GPS route if the request fails.
- **Anonymous Firebase auth means no built-in multi-device sync.** Every browser/device that opens the app signs in as a *different* anonymous user, so your phone and a desktop browser currently have separate data unless real authentication (e.g. email or Google sign-in) is added.
- **The whole UI re-renders on almost every state change.** This is simple and fast enough at this app's scale, but it's not how a larger app would typically be built — a framework with more targeted updates would scale better if PackTrax grew much further.
- **No automated tests.** Correctness currently relies on manual testing and the validation checks run after each change (the file is parsed for syntax errors before being shared).